Proyecto Final

Análisis de los hábitos de escucha en Spotify

Técnicas de Ciencia de Datos
Universidad de la Rioja

Realizado por:

Pablo Galilea Valverde
Jorge Jiménez Sáenz de Tejada

Logroño, Año 2026

Índice general

Planteamiento del problema

Contexto

Spotify es una de las plataformas de streaming musical con mayor número de usuarios a nivel mundial. A través de su API pública (*Spotify Web API*) permite, previa autenticación del propio usuario, consultar información agregada sobre sus hábitos de escucha: artistas más reproducidos, canciones favoritas, listas de reproducción, etc.

Este proyecto nace con el objetivo de aplicar el ciclo de vida de un proyecto de ciencia de datos —presentado en la primera sesión de la asignatura— sobre un conjunto de datos propio y personal: el historial de artistas más escuchados de los autores de este trabajo, enriquecido con información de género y de similitud obtenida de Last.fm y con una segunda medida de popularidad procedente de Deezer.

Análisis de negocio

Siguiendo el esquema de preguntas planteado en la asignatura para delimitar el objetivo de un proyecto de ciencia de datos (¿cuánto?, ¿qué categoría?, ¿qué grupo?, ¿es esto raro?, ¿qué opción se debe tomar?), este trabajo se sitúa principalmente en torno a las preguntas de **categorización**, **agrupamiento** y **comparación**:

- ¿Qué géneros musicales predominan en los gustos de cada usuario?
- ¿Es estable el gusto musical en el tiempo, o varía al comparar el corto plazo (*short_term*, últimas 4 semanas), el medio plazo (*medium_term*, últimos 6 meses) y el largo plazo (*long_term*, varios años)?
- ¿Existen agrupaciones naturales de artistas que permitan describir el perfil musical de forma más compacta que una simple lista?
- ¿Qué diferencias y similitudes existen entre los perfiles musicales de los dos autores del trabajo?
- ¿Se comportan los autores de forma parecida respecto a la popularidad de los artistas que escuchan?
- ¿Se puede estimar, para un artista ajeno al historial, si encaja con el gusto de un usuario a partir de la similitud con sus artistas favoritos?

Objetivo del proyecto

El objetivo principal es construir, a partir de datos personales obtenidos mediante la API de Spotify, un conjunto de datos estructurado que recoja los artistas más escuchados por cada autor en los tres rangos temporales disponibles, junto con los géneros musicales asociados a cada artista y otros atributos de popularidad y similitud. Sobre ese conjunto de datos se realiza:

- Un análisis descriptivo de la diversidad y la evolución del gusto musical (distribución de géneros, comparación entre rangos temporales y entre usuarios).
- Un análisis de compatibilidad entre los dos autores mediante métricas de similitud sobre artistas y géneros.
- Un análisis de popularidad, contrastando la posición en el ranking personal con los oyentes de Last.fm y los fans de Deezer.
- Una estimación de afinidad usuario–artista para artistas nuevos, basada en la similitud de contenido de Last.fm.
- Un *clustering* de artistas por sus géneros, en la línea de lo trabajado en la práctica de aprendizaje no supervisado de la asignatura.
- La visualización de los resultados mediante los tipos de gráficos estudiados en la asignatura, justificando en cada caso el tipo elegido.

Proyecto completado

El proyecto se entrega completo: extracción de datos para los dos autores, cuaderno de análisis ejecutado, enriquecimiento con Last.fm y Deezer, tratamiento documentado de valores nulos, análisis de compatibilidad, popularidad, afinidad y agrupamiento, diez figuras para el informe y conclusiones. Las únicas fuentes descartadas se justifican en su capítulo correspondiente (MusicBrainz) o vienen impuestas por la API de Spotify (*audio-features* y *related-artists*).

Datos recogidos

Fuentes de datos

Siguiendo la clasificación de fuentes públicas de datos vista en la asignatura, este proyecto utiliza tres fuentes encuadradas en la categoría de **APIs y datos en tiempo real / redes sociales**:

- **Spotify Web API**, a través de la librería `spotipy` para Python. Es la fuente primaria: proporciona, para el usuario autenticado, su lista de artistas más escuchados (`current_user_top_artists`) en tres rangos temporales.
- **Last.fm API**, fuente secundaria de tres tipos de información: los géneros más representativos de cada artista (`artist.gettoptags`), sus métricas de popularidad (`artist.getInfo`) y su vecindad de artistas similares con una puntuación de similitud (`artist.getSimilar`).
- **Deezer API pública**, utilizada como segunda medida de popularidad (número de fans y álbumes) y para contrastar la cobertura de Last.fm. No requiere autenticación.

A diferencia de los portales de datos abiertos gubernamentales o los repositorios de *Machine Learning* vistos en la Sesión 1 (Kaggle, UCI, datos.gob.es, etc.), estas fuentes requieren autenticación mediante credenciales de desarrollador (*API keys* / OAuth) y devuelven datos **personales y dinámicos**: el resultado depende del usuario autenticado y cambia con el tiempo según sus hábitos de escucha.

Se evaluó también **MusicBrainz** como fuente de metadatos canónicos (país de origen, año de actividad o identificadores estables). Se descartó porque, sin un análisis geográfico en el alcance del proyecto, no aportaba variables que respondieran a las preguntas planteadas y añadía una consulta adicional por artista.

Acceso a los datos

Autenticación en Spotify

El acceso a los datos de Spotify requiere registrar una aplicación en el panel de desarrolladores de Spotify para obtener un `Client ID` y un `Client Secret`, y autenticar al usuario mediante el flujo OAuth *Authorization Code Flow*, gestionado por `spotipy.oauth2.SpotifyOAuth`. El *scope* solicitado es `user-top-read`, que da acceso de solo lectura a los artistas y canciones más escuchados por el usuario. El token se guarda en una caché local que no se versiona.

Autenticación en Last.fm y Deezer

Last.fm se consulta con una `API key` simple enviada como parámetro en cada petición HTTP. Deezer expone un *endpoint* de búsqueda público que no requiere credenciales. En ambos casos se aplica una pausa entre peticiones y reintentos con espera creciente ante errores temporales o límites de frecuencia.

Gestión de credenciales

Las credenciales (`SPOTIPY_CLIENT_ID`, `SPOTIPY_CLIENT_SECRET`, `SPOTIPY_REDIRECT_URI` y `LASTFM_API_KEY`) se almacenan en un fichero `.env` local, cargado mediante la librería `python-dotenv`, y no se incluyen en el repositorio del proyecto por motivos de seguridad. Cada autor ejecuta la extracción con su propia cuenta de Spotify; las credenciales de la aplicación pueden compartirse, pero el token de acceso es personal.

Datos obtenidos

Para cada uno de los dos autores del trabajo se ejecuta el proceso de extracción, obteniendo un listado de hasta 50 artistas por cada uno de los tres rangos temporales que ofrece la API de Spotify:

Rango (API)	**Significado**	**Etiqueta usada**
`short_term`	Últimas 4 semanas	`ultimas_4_semanas`
`medium_term`	Últimos 6 meses	`ultimos_6_meses`
`long_term`	Varios años (histórico acumulado)	`varios_anios`

Cuadro 1: Rangos temporales proporcionados por la API de Spotify para el ranking de artistas más escuchados.

Para cada artista se recogen los siguientes campos:

Campo	**Descripción**	**Fuente**
`rango`	Rango temporal al que pertenece la entrada (ver Tabla 1)	Spotify
`posicion`	Posición en el ranking dentro de su rango (1–50)	Spotify
`artista`	Nombre del artista	Spotify
`generos`	Hasta 4 etiquetas de género, separadas por `;`	Last.fm
`spotify_url`	Enlace al perfil del artista en Spotify	Spotify

Cuadro 2: Esquema del conjunto de datos recogido por autor (fichero `top_artistas*.csv`).

Cada autor aporta un máximo de 150 filas (50 por rango). El número de artistas únicos es inferior porque un mismo artista puede aparecer en varios rangos; esa repetición es en sí misma una variable de interés para el análisis de la estabilidad del gusto musical. La extracción de ambos autores se realiza en una única fecha, registrada junto con los recuentos en `resultados/extraccion_meta.json`.

Sobre esa base, la tabla de artistas se amplía con los campos de la Tabla 3.

Cuadro 3: Campos adicionales incorporados por artista.

Campo	**Fuente**	**Descripción**
`listeners`	Last.fm	Número de oyentes únicos del artista
`playcount`	Last.fm	Reproducciones totales registradas
`mbid`	Last.fm	Identificador MusicBrainz del artista
`similar_top`	Last.fm	Artista más similar y su *match score*
`n_similares`	Last.fm	Número de vecinos similares recuperados
`deezer_fans`	Deezer	Número de fans en Deezer
`deezer_nb_album`	Deezer	Número de álbumes en Deezer

Problemas de calidad e integración detectados

- **Cobertura incompleta de géneros**: no todos los artistas tienen etiquetas en Last.fm, especialmente bandas sonoras de videojuegos o artistas japoneses minoritarios (p. ej. `ATLUS GAME MUSIC` o compositores con pocos oyentes en Last.fm), que aparecen sin género asignado.
- **Etiquetas de Last.fm ruidosas**: Last.fm permite etiquetas libres de usuario, por lo que junto a géneros reales aparecen etiquetas de nacionalidad (`spanish`, `japanese`, `taiwan`) o metaetiquetas sin valor musical (`seen live`, `favorites`, `my top songs`). Se ha aplicado un filtro manual (`IGNORAR`) para excluir las etiquetas más comunes de este tipo, aunque no es exhaustivo.
- **Cobertura desigual entre fuentes**: los artistas de nicho tienen menos probabilidades de estar dados de alta en Last.fm y Deezer, por lo que los valores nulos no se distribuyen al azar. Su cuantificación y tratamiento se detallan en el Capítulo 0.13.
- **Redundancia entre rangos**: un mismo artista puede aparecer hasta tres veces (una por rango), por lo que el fichero combinado no está en forma de tabla *artista–atributos* única, sino en formato largo (*long format*), con una fila por combinación artista-rango.
- **Normalización de nombres**: los nombres de artistas provienen directamente de Spotify y se usan tal cual para consultar Last.fm y Deezer; en caso de alias distintos entre plataformas, la búsqueda puede fallar silenciosamente (se guarda cadena vacía o valor nulo).

Tratamiento de los datos

Herramientas y librerías utilizadas

El proyecto se ha desarrollado íntegramente en Python, dentro de un entorno virtual reproducible descrito en `requirements.txt`. El trabajo se reparte entre dos cuadernos Jupyter y un módulo de funciones:

- `TrabajoTecnicasDeCienciaDeDatos.ipynb`: extracción de los artistas más escuchados y de sus géneros.
- `AnalisisTecnicasDeCienciaDeDatos.ipynb`: consolidación, enriquecimiento, análisis y generación de figuras.
- `spotify_analisis.py`: funciones puras reutilizables por ambos cuadernos.

La Tabla 4 recoge las librerías empleadas y su función.

Librería	**Uso en el proyecto**
`spotipy`	Cliente de la Spotify Web API; gestiona el flujo OAuth y las peticiones a `current_user_top_artists`.
`requests`	Peticiones HTTP a Last.fm y Deezer, con reintentos y control de errores.
`pandas`	Consolidación en formato largo, tablas derivadas, agregaciones y exportación.
`numpy`	Operaciones vectoriales y cálculo de la entropía.
`scipy`	Correlación de Spearman y prueba U de Mann-Whitney.
`scikit-learn`	K-Means, coeficiente de silueta y proyección PCA.
`matplotlib` y `seaborn`	Figuras del informe, con estilo homogéneo y 300 dpi.
`python-dotenv`	Carga de credenciales desde el fichero local `.env`.
`jupyterlab`	Ejecución interactiva de los cuadernos.
`time` y `json` (estándar)	Pausas entre peticiones y caché persistente de enriquecimiento.

Cuadro 4: Librerías empleadas en el proyecto.

Formato de los datos

Se ha optado por **CSV** (valores separados por comas) para los datos de extracción y los agregados, codificado en `utf-8-sig` para preservar correctamente los caracteres no latinos

presentes en los nombres de artistas (japonés, coreano, etc.) y mantener compatibilidad con Excel. Es un formato estructurado y legible, adecuado para un volumen de decenas o cientos de filas que se procesa con `pandas`; alternativas orientadas a grandes volúmenes o datos jerárquicos, como HDF5 o Parquet, no aportan ventajas con este tamaño y dificultan la inspección manual.

La **caché de enriquecimiento** se guarda en **JSON** porque su estructura es anidada (por artista, con listas de similares) y porque permite reanudar el proceso sin repetir llamadas a las APIs. Los metadatos de la extracción y las métricas finales también se exportan en JSON, que es directamente consumible por el informe y por cualquier validación posterior.

Proceso de obtención de los datos base

El proceso implementado en el cuaderno de extracción sigue estos pasos:

1. **Autenticación**: se cargan las credenciales desde `.env` y se inicializa el cliente `spotipy.Spotify` con el *scope* `user-top-read`. El token se cachea localmente.
2. **Descarga de artistas por rango**: para cada uno de los tres rangos temporales (Tabla 1), se solicita el *top* 50 de artistas más escuchados (`current_user_top_artists`) y se añade cada uno a una lista de registros con su posición, nombre y enlace de Spotify.
3. **Deduplicación antes de consultar géneros**: dado que un mismo artista puede repetirse en varios rangos, se extrae el conjunto de artistas *únicos* antes de consultar Last.fm, para minimizar el número de peticiones y respetar sus límites de uso.
4. **Consulta de géneros en Last.fm**: para cada artista único se solicita `artist.gettoptags`, que devuelve etiquetas con un peso (*count*) entre 0 y 100. Se seleccionan como máximo 4 etiquetas con peso igual o superior a 20, descartando las etiquetas no musicales recogidas en el conjunto `IGNORAR` (nacionalidades, tipo de voz, metaetiquetas).
5. **Combinación**: los géneros se incorporan al `DataFrame` mediante un mapeo artista $\rightarrow$ géneros, evitando repetir la consulta para un artista presente en varios rangos.
6. **Exportación**: el resultado se guarda como `top_artistas.csv` o `top_artistasJorge.csv` según el autor, con las columnas `rango`, `posicion`, `artista`, `generos` y `spotify_url` (Tabla 2).

Proceso de enriquecimiento

Sobre el conjunto de artistas únicos de ambos autores, el cuaderno de análisis realiza tres consultas por artista:

1. **Last.fm** `artist.getInfo`: oyentes, reproducciones e identificador MusicBrainz.
2. **Last.fm** `artist.getSimilar`: hasta 20 artistas similares con su *match score*.
3. **Deezer**: búsqueda pública del artista para obtener fans y número de álbumes.

Las respuestas se guardan en `cache/enriquecimiento.json` con una clave por artista y fuente. Antes de cada llamada se comprueba la caché, de modo que una segunda ejecución no repite peticiones. Se aplica una pausa de 0,25 s en Last.fm y 0,1 s en Deezer, y se reintenta con espera creciente ante errores 429 o 5xx. Los artistas no encontrados se almacenan con valores nulos y la bandera `fuente_encontrada=False` para no reintentarlos indefinidamente. La URL de las peticiones, que contiene la *API key*, nunca se escribe en los registros.

Privacidad y reproducibilidad

Los datos de partida son personales: identifican los gustos musicales de los dos autores. Por ello:

- Los CSV crudos (`top_artistas*.csv`), el fichero `.env`, la caché de tokens (`.cache`) y la caché de enriquecimiento no se versionan; están excluidos en `.gitignore`.
- Solo se publican agregados sin nombres de artistas (`resultados/*.csv` y `resultados/metricas.j` y las figuras, que tampoco etiquetan artistas individuales salvo en los análisis agregados.
- Cada autor ejecuta la extracción con su propia cuenta; el token de acceso es intransferible aunque las credenciales de la aplicación se compartan.
- La extracción de ambos autores se realiza el mismo día y la fecha se registra en `resultados/extraccion_meta.json` para acotar la deriva temporal.
- La semilla de los procesos aleatorios es fija (42) y los parámetros relevantes (número de similares, soporte mínimo de género, mínimo de artistas por grupo para imputar y rango de k) están definidos como constantes del módulo.

Decisiones de diseño

- Se ha mantenido una primera versión simplificada del script (sin consulta a Last.fm) junto a la versión final, documentando así la evolución del proceso: primero se validó el acceso a la API de Spotify y, una vez funcionando, se añadió el enriquecimiento con géneros.
- El límite de **4 géneros por artista** y el **umbral mínimo de peso 20** se fijaron para equilibrar riqueza informativa y precisión, evitando etiquetas minoritarias o poco representativas.
- La lógica de análisis se extrajo a un módulo independiente para que el cuaderno sea legible y para poder reutilizar las mismas funciones con otros usuarios o con una nueva extracción.
- Cada autor ejecuta el proceso sobre su propia cuenta, generando un CSV por persona, lo que permite comparar los perfiles musicales sin mezclar datos personales en el repositorio.

Diseño del modelo de datos

Este capítulo describe el modelo de datos construido para el análisis y las decisiones de tratamiento que lo acompañan. Todas las transformaciones están implementadas como funciones puras en el módulo `spotify_analisis.py`, de modo que el cuaderno `AnalisisTecnicasDeCienciaDeDatos.ipynb` se limita a orquestarlas y a presentar los resultados. La separación entre lógica y presentación permite reproducir el análisis completo con un único *Restart & Run All* y reutilizar las mismas funciones con los datos de cualquier otro usuario.

Estructura de partida y consolidación

El conjunto de datos de partida (Capítulo 0.3) está en **formato largo**: una fila por combinación (usuario, rango, posición). Sobre él se construyen las siguientes vistas:

- **Tabla larga consolidada**: unión de los CSV de ambos autores con las columnas originales más `usuario` y `rango_orden` (1 = últimas 4 semanas, 2 = últimos 6 meses, 3 = varios años). La posición se convierte a entero y la columna `generos` se normaliza a cadena vacía cuando falta.
- **Tabla artista–género**: una fila por cada género individual de cada artista, separando la cadena `generos` por `;`. Los géneros se normalizan a minúsculas y se deduplican dentro de cada artista, de forma que un artista que aparezca en varios rangos no infla el recuento.
- **Resumen por usuario y rango**: número de artistas distintos, artistas con al menos un género, porcentaje de cobertura y número de géneros distintos.
- **Tabla combinada**: permite comparar directamente los perfiles de ambos autores, que es la base de la compatibilidad del §0.18.1.

La Tabla 5 resume el grano y el uso de cada tabla derivada.

Cuadro 5: Tablas derivadas del modelo de datos.

Tabla	**Grano**	**Uso principal**
`datos_largo`	usuario × rango × posición	Base del análisis descriptivo y de la estabilidad
`artista_genero`	artista × género	Frecuencias de género y matriz binaria para el agrupamiento
`perfil_usuario_rango`	usuario × rango	Cobertura y diversidad por rango
`artistas`	artista único	Atributos enriquecidos y banderas de imputación
`similitudes`	artista × similar	Candidatos y puntuación de afinidad
`compatibilidad`	rango (incluido global)	Comparación entre autores
`afinidad`	candidato × usuario	Ranking de artistas nuevos por usuario
`clusters`	artista	Asignación de cada artista a un grupo

Enriquecimiento con fuentes adicionales

El modelo no se limita a los géneros: cada artista único se enriquece con atributos de popularidad y de similitud procedentes de Last.fm y Deezer. La Tabla 6 recoge los campos incorporados y su finalidad.

Cuadro 6: Campos incorporados en la tabla de artistas.

Campo	**Fuente**	**Uso**
`listeners`, `playcount`, `mbid`	Last.fm (`artist.getInfo`)	Popularidad general del artista
`similar_top`, `match_top`, `n_similares`	Last.fm (`artist.getSimilar`)	Vecindad de similitud para la afinidad
`deezer_fans`, `deezer_nb_album`	Deezer (búsqueda pública)	Contraste de popularidad y generación de nulos reales

Las consultas se realizan una sola vez por artista y se guardan en una caché JSON local (`cache/enriquecimiento.json`), de modo que una segunda ejecución no repite llamadas. Se aplica una pausa entre peticiones y reintentos con espera creciente ante errores temporales o límites de frecuencia. Los artistas no encontrados se registran con valores nulos y la bandera correspondiente, en lugar de reintentarse indefinidamente.

Se evaluó también **MusicBrainz** como fuente de metadatos canónicos (país, año de actividad, identificadores). Se descartó porque, sin un mapa geográfico en el alcance del proyecto, no aportaba variables que mejorasen las preguntas planteadas y añadía una consulta adicional por artista.

Cabe recordar que la *Spotify Web API* dejó de ofrecer los *endpoints* de `audio-features` y `related-artists` para aplicaciones de desarrollador nuevas, por lo que las características de audio y los artistas relacionados de Spotify no están disponibles como fuente; de ahí que Last.fm cubra la señal de similitud entre artistas.

Tratamiento de valores nulos

Los nulos aparecen en dos niveles: géneros vacíos en el CSV de extracción y atributos ausentes en las fuentes externas (artistas de nicho, bandas sonoras o proyectos minoritarios que no están dados de alta en todas las plataformas). El tratamiento es deliberadamente distinto según el tipo de variable:

- **Popularidad** (`listeners`, `playcount`, `deezer_fans`): se cuantifica el porcentaje de nulos por columna y por fuente y se imputa la mediana del género principal cuando ese grupo tiene al menos cinco artistas con dato; en caso contrario se usa la mediana global. Cada imputación queda marcada con una bandera `imputado_*`.
- **Similitud y género**: no se imputan. Un artista sin vecinos similares o sin etiquetas no tiene un sustituto razonable y tratarlo como si lo tuviera introduciría una señal artificial. Se documenta qué artistas quedan fuera de cada análisis.
- **Correlaciones de popularidad**: se calculan sobre casos completos, nunca sobre valores imputados, para no inflar artificialmente la correlación.

La Figura **??** muestra la cobertura resultante y la Tabla 7 resume las reglas aplicadas.

Cuadro 7: Reglas de tratamiento de valores nulos.

Variable	**¿Se imputa?**	**Criterio**
`listeners`	Sí	Mediana por género principal (grupo ≥ 5); si no, mediana global
`playcount`	Sí	Mismo criterio que `listeners`
`deezer_fans`	Sí	Mismo criterio que `listeners`
Similitud	No	Información ausente real
Género	No	Información ausente real

Características y variables del modelo

A partir de las tablas anteriores se derivan las variables que alimentan el análisis:

- **Frecuencia de género**: número de artistas distintos asociados a cada género, global y por usuario, para evitar que un artista repetido en tres rangos cuente tres veces.
- **Diversidad**: número de artistas, número de géneros, entropía de Shannon (base 2) de la distribución de géneros y cuota del género principal.
- **Estabilidad temporal**: similitud entre los conjuntos de artistas de cada par de rangos (Jaccard) y entre los vectores de frecuencia de género (coseno).
- **Compatibilidad entre usuarios**: las mismas dos métricas aplicadas a los perfiles de ambos autores, por rango y en el conjunto de los tres rangos.
- **Popularidad frente a ranking**: correlación de Spearman entre la posición en el ranking y el logaritmo de los oyentes de Last.fm, por usuario y rango.
- **Afinidad usuario–artista**: puntuación continua en $[0, 1]$ para artistas ajenos al historial, descrita en el §**??**.
- **Grupo de artista**: asignación de cada artista a un clúster por su vector binario de géneros, descrita en el §**??**.

Métodos de análisis

Compatibilidad y estabilidad

Para dos conjuntos de artistas A y B se usa el coeficiente de Jaccard:

$$J(A, B) = \frac{|A \cap B|}{|A \cup B|},$$

que vale 1 cuando los conjuntos coinciden y 0 cuando son disjuntos. Para los géneros se construye un vector de frecuencias por usuario y rango, alineado sobre la unión de géneros, y se calcula la similitud del coseno:

$$\cos(\mathbf{u}, \mathbf{v}) = \frac{\mathbf{u} \cdot \mathbf{v}}{\|\mathbf{u}\| \, \|\mathbf{v}\|}.$$

Ambas métricas se calculan para cada par de rangos (estabilidad) y entre usuarios (compatibilidad), incluyendo una fila global que agrega los tres rangos. El resultado se interpreta de forma descriptiva: con dos usuarios no se plantea contraste de hipótesis ni generalización poblacional.

Afinidad usuario–artista

No existe una fuente fiable de ejemplos negativos (Spotify no expone artistas bloqueados u ocultados), por lo que se descarta un clasificador supervisado y se opta por un enfoque de **similitud de contenido**. Para cada usuario u se define el peso de un artista a de su historial como

$$w_u(a) = \frac{51 - \text{mín}_r \, \text{posición}_r(a)}{50},$$

de modo que el primer puesto de cualquier rango pesa aproximadamente 1 y el puesto 50 pesa 0,02. Para un artista candidato c con vecinos similares $s \in \text{Sim}(c)$ y puntuaciones de similitud $\text{match}(c, s)$, la afinidad estimada es

$$S(c, u) = \frac{\sum_{s \in \text{Sim}(c)} \text{match}(c, s)\, w_u(s)}{\sum_{s \in \text{Sim}(c)} \text{match}(c, s)}.$$

La puntuación es nula si el artista no tiene vecinos. Los candidatos se generan a partir de los diez artistas mejor situados de cada usuario, se excluyen los que ya aparecen en cualquier historial y se conservan los veinte con mayor similitud media.

Como comprobación descriptiva se aplica un esquema *leave-one-out*: se oculta cada artista del top-10, se recalcula su puntuación sin él y se compara con la de los candidatos-control mediante medianas y la prueba U de Mann-Whitney. No es una evaluación supervisada ni una medida de acierto predictivo; solo indica si la señal ordena razonablemente los artistas conocidos por encima de los ajenos.

Agrupamiento de artistas

Cada artista se representa por un vector binario de géneros (1 si el artista tiene ese género). Se conservan los géneros presentes en al menos dos artistas y los artistas con al menos un género; las exclusiones se documentan. Sobre esa matriz se aplica K-Means con semilla fija (42) y diez inicializaciones. El número de grupos se elige en el rango $k \in [2, 8]$ combinando el método del codo (inercia) y el coeficiente de silueta, prefiriendo el k menor cuando las diferencias de silueta son inferiores a 0,02. Los grupos se etiquetan por los tres géneros con mayor prevalencia relativa y se proyectan en dos dimensiones mediante PCA, que es solo una ayuda visual: el agrupamiento se realiza en el espacio completo de géneros.

Decisiones de implementación y reproducibilidad

- **Semilla fija** (42) en K-Means y en cualquier proceso aleatorio.
- **Caché persistente** de enriquecimiento para evitar llamadas repetidas y garantizar que dos ejecuciones producen los mismos datos.

- **Credenciales fuera del repositorio**: el fichero `.env` y la caché de tokens no se versionan; los CSV con nombres de artistas tampoco.
- **Salidas separadas**: las tablas con nombres de artistas quedan en local y solo se publican agregados y figuras sin etiquetas identificativas.
- **Parámetros explícitos**: número de similares (20), mínimo de soporte de género (2), mínimo de artistas por grupo para imputar (5) y rango de k (2–8) están definidos como constantes del módulo.

Limitaciones del modelo

- **Dos usuarios**: los índices de compatibilidad describen a estos dos autores, no a una población.
- **Etiquetas de género colaborativas**: Last.fm es una folksonomía; las etiquetas pueden ser ruidosas, redundantes o mezclar género con país o época.
- **Datos dinámicos**: el top de Spotify cambia con el tiempo; ambos CSV deben extraerse el mismo día y la fecha se registra con los metadatos.
- **Cobertura desigual**: los artistas de nicho tienen menos probabilidades de estar en Last.fm o Deezer, lo que puede sesgar las comparaciones de popularidad.
- **Sin características de audio**: la retirada de `audio-features` impide usar *danceability*, *energy* u otras variables acústicas.

Resultados y visualización

Pendiente de completar

Este capítulo recogerá las visualizaciones y los resultados numéricos obtenidos al ejecutar el análisis sobre el modelo de datos descrito en el Capítulo 0.13. Se deja planteado el plan de visualizaciones, siguiendo los criterios de audiencia, objetivo y tipo de gráfico vistos en la Sesión 6 de la asignatura (Visualización de datos).

Plan de visualizaciones

Pregunta	**Gráfico previsto**	**Referencia**
¿Qué géneros predominan por usuario?	Gráfico de barras horizontales (*top* N géneros por frecuencia)	§0.17
¿Cómo se reparten los géneros en conjunto?	Gráfico circular o de barras apiladas, comparando ambos usuarios	§0.17
¿Cambia el gusto musical según el rango temporal?	Gráfico de barras agrupado por rango (`countplot` por rango y género)	Tabla 1
¿Cuántos artistas se repiten entre rangos?	Diagrama de Venn o tabla cruzada de coincidencias	§0.17
¿Qué tan compatibles son los dos usuarios?	Gráfico de barras o mapa de calor con el índice de compatibilidad por rango	§0.18.1
¿Artistas populares o de nicho?	*Scatter plot* de posición en el *ranking* frente a popularidad (Last.fm)	§0.15
Afinidad estimada con un artista nuevo	Gráfico de barras con el grado de solapamiento con artistas similares	§??
Agrupamiento de artistas/usuarios por género (opcional)	*Scatter plot* del resultado del *clustering*, con baricentros	§??

Cuadro 8: Plan de visualizaciones previsto para el análisis de resultados.

Figuras y tablas de resultados

(Pendiente de incorporar las figuras generadas a partir del análisis sobre `top_artistas.csv` *y* `top_artistasJorge.csv`*.)*

Conclusiones

Pendiente de completar

Las conclusiones definitivas se redactarán una vez completado el análisis y la visualización de resultados (Capítulos 0.13 y **??**). A continuación se dejan algunas conclusiones preliminares, centradas en el propio proceso de obtención de datos.

Conclusiones sobre el proceso de obtención de datos

- Combinar dos APIs externas (Spotify y Last.fm) permite enriquecer un conjunto de datos personal con información que ninguna de las dos fuentes ofrece de forma completa por sí sola, a costa de introducir ruido y huecos (artistas sin género) propios de combinar fuentes de distinta calidad y cobertura.
- El uso de una caché en memoria por artista único, en lugar de consultar Last.fm una vez por cada fila del conjunto de datos, resultó clave para mantener el proceso dentro de los límites de peticiones de la API externa.
- Trabajar con datos personales y dinámicos (dependientes del usuario autenticado y del momento de la consulta) implica que los resultados de este proyecto no son directamente reproducibles por un tercero sin sus propias credenciales y su propio historial de escucha, a diferencia de un conjunto de datos abierto y estático.

Conclusiones del análisis

(Pendiente de redactar tras completar los Capítulos 0.13 y **??***: diversidad de géneros por usuario, estabilidad del gusto musical entre rangos temporales, grado de afinidad musical entre los dos autores y, en su caso, interpretación de los grupos obtenidos mediante* clustering*.)*

Limitaciones y trabajo futuro

- El filtro de etiquetas no musicales de Last.fm (`IGNORAR`) es manual y no exhaustivo; podría mejorarse con una lista de géneros de referencia o un modelo de clasificación de etiquetas.
- El análisis se limita a dos usuarios; las conclusiones sobre similitud/diversidad musical no son generalizables más allá de los propios autores.
- La línea de afinidad usuario–artista (§**??**) se apoya en similitud de contenido precisamente porque no existe una fuente fiable de ejemplos negativos (Spotify no expone artistas bloqueados/ocultados); un clasificador supervisado real de "le gusta / no le gusta" quedaría fuera del alcance de los datos disponibles.

- La *Spotify Web API* ya no ofrece, para aplicaciones nuevas, los *endpoints* de `audio-features` ni `related-artists` (restringidos desde noviembre de 2024), lo que limita el enriquecimiento a lo que puedan aportar fuentes externas como Last.fm, Deezer o MusicBrainz (§0.15).
- Como ampliación, podría incorporarse también el historial de canciones más escuchadas (no solo artistas) además de los atributos de popularidad y similitud ya planteados.